

DNS.SetSolarWindsRecordToActiveServer.ps1

PowerShell script - no load balancer and no SNMP. It is told which SolarWinds server has become active and points the DNS record straight at it.

291 lines · Scripts/DNS.SetSolarWindsRecordToActiveServer.ps1

```
1 #Requires -Version 5.1
2
3 <#
4 .SYNOPSIS
5     Points the SolarWinds DNS record at the SolarWinds server that has just
6     become active. No load balancer, no web servers, no SNMP.
7
8 .DESCRIPTION
9     This is the Alteon-free variant of DNS.SetSolarWindsRecordToActiveLB.v2.ps1.
10    That script asks a Radware load balancer over SNMP how the web servers
11    behind it are doing, and picks a load balancer accordingly. This one does
12    not ask anything: it is told which SolarWinds server is now active, and
13    points the record straight at it.
14
15    Being told is the point. SolarWinds decided which server is active, so
16    SolarWinds is the authority on it. Anything the script worked out for
17    itself would at best reproduce that answer and at worst contradict it.
18
19    It is meant to run as an alert action on the server that has just become
20    active, with that server's address passed in -ActiveServer. Nothing is
21    written when the record already holds the wanted address, so it is also
22    safe to run on a schedule.
23
24 .PARAMETER ActiveServer
25     The address of the SolarWinds server that is now active, and the address
26     the record will be pointed at. It has to match either PrimaryDcSolarWindsIp
27     or SecondaryDcSolarWindsIp, otherwise the script fails instead of quietly
28     doing nothing.
29
30 .PARAMETER DnsServer
31     The DNS server to read the record from and write it back to. It has to hold
32     a writable copy of the zone. When it is left out the script walks the DNS
33     servers configured on this host and takes the first one that answers a ping.
34
35 .PARAMETER LogMaxBytes
36     The size at which the log is rotated. Rotation keeps LogKeep older
37     generations beside it, so the space the logging can occupy is bounded at
38     roughly LogMaxBytes times LogKeep plus one.
39
40 .PARAMETER LogKeep
41     How many rotated logs to keep. Zero keeps none: the log starts over
42     instead.
43
44 .PARAMETER RecordTtlSeconds
45     When greater than zero, the TTL written onto the record. A failover only
46     takes effect once the old TTL has expired everywhere, so a low value (30-60)
47     is worth setting here.
48
49 .EXAMPLE
50     .\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1 -WhatIf
51
52     Shows what would change without touching DNS.
53
54 .EXAMPLE
55     .\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.20.1
56
57 .NOTES
58     Exit codes: 0 done (changed or nothing to change), 1 failed.
```

```

59
60 Call it with -File, not -Command:
61
62     powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1
63
64 A parameter that fails validation stops the script before it runs, and only
65 -File turns that into exit code 1. Under -Command the caller reads a stale
66 $LASTEXITCODE instead, so a typo in the arguments looks like a clean run.
67
68 The record points at the SolarWinds server itself. If a load balancer sits
69 in front of SolarWinds in your environment, this is the wrong script for it:
70 use DNS.SetSolarWindsRecordToActiveLB.v2.ps1, which picks between the two
71 load balancers on the health of the web servers behind them.
72 #>
73
74 [CmdletBinding(SupportsShouldProcess)]
75 param(
76     [Parameter(Mandatory, Position = 0)]
77     [ValidateNotNullOrEmpty()]
78     [string] $ActiveServer,
79
80     [ValidateNotNullOrEmpty()]
81     [string] $RecordName = 'OurSolar',
82
83     [ValidateNotNullOrEmpty()]
84     [string] $ZoneName = 'OurZone',
85
86     [string] $DnsServer,
87
88     [ValidateNotNullOrEmpty()]
89     [string] $PrimaryDcSolarWindsIp = '10.10.10.1',
90
91     [ValidateNotNullOrEmpty()]
92     [string] $SecondaryDcSolarWindsIp = '10.10.20.1',
93
94     [ValidateRange(0, 86400)]
95     [int] $RecordTtlSeconds = 0,
96
97     [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\dns-active-server.log'),
98
99     [ValidateRange(4KB, 100MB)]
100    [int] $LogMaxBytes = 1MB,
101
102    [ValidateRange(0, 20)]
103    [int] $LogKeep = 3
104)
105
106 Set-StrictMode -Version Latest
107 $ErrorActionPreference = 'Stop'
108
109 function Invoke-LogRotation {
110     # Renames the log out of the way once it reaches -LogMaxBytes, keeping
111     # -LogKeep older generations: .log -> .log.1, .log.1 -> .log.2, and so on,
112     # with the oldest dropped. Run on a schedule the log would otherwise grow
113     # without bound.
114     param(
115         [Parameter(Mandatory)][string] $Path
116     )
117
118     if (-not (Test-Path -LiteralPath $Path)) { return }
119     if ((Get-Item -LiteralPath $Path).Length -lt $script:LogMaxBytes) { return }
120
121     # Nothing to keep: the log simply starts over.
122     if ($script:LogKeep -lt 1) {
123         Remove-Item -LiteralPath $Path -Force -WhatIf:$false -Confirm:$false

```

```

124     return
125 }
126
127 # Oldest generation falls off the end.
128 $oldest = "$Path.$($script:LogKeep)"
129 if (Test-Path -LiteralPath $oldest) {
130     Remove-Item -LiteralPath $oldest -Force -WhatIf:$false -Confirm:$false
131 }
132
133 # Shift the rest down, highest first so nothing is overwritten on the way.
134 for ($i = $script:LogKeep - 1; $i -ge 1; $i--) {
135     $from = "$Path.$i"
136     if (Test-Path -LiteralPath $from) {
137         Move-Item -LiteralPath $from -Destination "$Path.($i + 1)" -Force -WhatIf:$false -Confirm:$false
138     }
139 }
140
141 Move-Item -LiteralPath $Path -Destination "$Path.1" -Force -WhatIf:$false -Confirm:$false
142 }
143
144 function Write-Log {
145     param(
146         [Parameter(Mandatory)]
147         [AllowEmptyString()]
148         [string] $Message,
149
150         [ValidateSet('INFO', 'WARN', 'ERROR')]
151         [string] $Level = 'INFO'
152     )
153
154     $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
155     Write-Information -MessageData $line -InformationAction Continue
156
157     # -WhatIf:$false so that a -WhatIf run still leaves a trace of what it
158     # decided; the log is a record, never one of the changes being previewed.
159     try {
160         $directory = Split-Path -Path $script:LogPath -Parent
161         if ($directory -and -not (Test-Path -LiteralPath $directory)) {
162             New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false | Out-Null
163         }
164         Invoke-LogRotation -Path $script:LogPath
165         Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false
166     }
167     catch {
168         Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
169     }
170 }
171
172 function Resolve-DnsServerAddress {
173     # Walks the DNS servers configured on this host and returns the first one
174     # that answers a ping. Only adapters with IP enabled are looked at, so
175     # disconnected or disabled NICs cannot contribute a stale address.
176     [OutputType([string])]
177     param()
178
179     $adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter 'IPEnabled = True'
180
181     foreach ($adapter in $adapters) {
182         foreach ($address in $adapter.DNSServerSearchOrder) {
183             if (Test-Connection -ComputerName $address -Count 1 -Quiet -ErrorAction SilentlyContinue) {
184                 Write-Log "Using DNS server $address, configured on '$($adapter.Description)'."
185                 return [string]$address
186             }
187         }
188         Write-Log "DNS server $address did not answer a ping, trying the next one." -Level WARN
189     }
190 }

```

```

189     }
190 }
191
192 throw 'None of the DNS servers configured on this host answered a ping. Pass -DnsServer to name one
explicitly.'
193}
194
195function Get-SolarWindsRecord {
196    param(
197        [Parameter(Mandatory)][string] $Server
198    )
199
200    $records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -Name $RecordName -
RRType A)
201
202    if ($records.Count -eq 0) {
203        throw "Zone '$ZoneName' on $Server holds no A record named '$RecordName'."
204    }
205
206    if ($records.Count -gt 1) {
207        throw "Zone '$ZoneName' on $Server holds $($records.Count) A records named '$RecordName'. This script
handles a single A record, so the duplicates have to be resolved first."
208    }
209
210    return $records[0]
211}
212
213function Set-SolarWindsRecord {
214    # Set-DnsServerResourceRecord needs the untouched record and the edited copy
215    # as two separate objects, hence the Clone.
216    param(
217        [Parameter(Mandatory)][string] $Server,
218        [Parameter(Mandatory)][string] $TargetIp
219    )
220
221    $current = Get-SolarWindsRecord -Server $Server
222    $updated = $current.Clone()
223    $updated.RecordData.IPv4Address = [System.Net.IPAddress]::Parse($TargetIp)
224
225    if ($RecordTtlSeconds -gt 0) {
226        $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
227    }
228
229    Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -OldInputObject $current -
NewInputObject $updated
230
231    $after = Get-SolarWindsRecord -Server $Server
232    $afterIp = $after.RecordData.IPv4Address.IPAddressToString
233
234    if ($afterIp -ne $TargetIp) {
235        throw "The record was written but still reads $afterIp instead of $TargetIp."
236    }
237}
238
239try {
240    Write-Log "===== Run started for active server $ActiveServer ====="
241
242    # A SolarWinds alert action fills this in from a macro, and a macro can
243    # hand over its value padded with whitespace. Trimming here means a stray
244    # space cannot stop a failover; without it the address matches neither
245    # server and the run fails for no good reason.
246    $ActiveServer = $ActiveServer.Trim()
247
248    # The address is checked against the two servers the script knows rather
249    # than used as given, so a typo in the alert action cannot point the record

```

```

250 # at something that was never a SolarWinds server.
251 if ($ActiveServer -eq $PrimaryDcSolarWindsIp) {
252     $activeName = 'PrimaryDC'
253 }
254 elseif ($ActiveServer -eq $SecondaryDcSolarWindsIp) {
255     $activeName = 'SecondaryDC'
256 }
257 else {
258     throw "ActiveServer '$ActiveServer' is neither the PrimaryDC SolarWinds server
($PrimaryDcSolarWindsIp) nor the SecondaryDC one ($SecondaryDcSolarWindsIp)."
259 }
260
261 Write-Log "$activeName is active, so the record should point at $ActiveServer."
262
263 $dnsServerAddress = if ($DnsServer) { $DnsServer } else { Resolve-DnsServerAddress }
264
265 $record = Get-SolarWindsRecord -Server $dnsServerAddress
266 $currentIp = $record.RecordData.IPv4Address.IPAddressToString
267 Write-Log "$RecordName.$ZoneName currently points at $currentIp with a TTL of $($record.TimeToLive)."
268
269 if ($currentIp -eq $ActiveServer) {
270     Write-Log "Nothing to do, the record already points at the $activeName server ($ActiveServer)."
271     exit 0
272 }
273
274 $target = "$RecordName.$ZoneName on $dnsServerAddress"
275 $action = "Repoint from $currentIp to $ActiveServer, the $activeName server"
276
277 if (-not $PSCmdlet.ShouldProcess($target, $action)) {
278     Write-Log "WhatIf: would have repointed $RecordName.$ZoneName from $currentIp to $ActiveServer
($activeName server)."
279     exit 0
280 }
281
282 Set-SolarWindsRecord -Server $dnsServerAddress -TargetIp $ActiveServer
283 Write-Log "Repointed $RecordName.$ZoneName from $currentIp to $ActiveServer, the $activeName server."
284 exit 0
285}
286catch {
287    Write-Log "Run failed: $($_.Exception.Message)" -Level ERROR
288    Write-Log $_.ScriptStackTrace -Level ERROR
289    Write-Error -Message $_.Exception.Message -ErrorAction Continue
290    exit 1
291}

```